Логистика WB: ручное управление = потери

1000+

Тысячи заказов

Каждые 30 минут — новая волна заказов, требующих немедленной маршрутизации

Ручные решения

Диспетчер решает вручную: когда отправлять, какой транспорт назначить

Результат — потери

Переполненные склады, нарушенные SLA, пустые рейсы

ОБЪЁМ → РУЧНОЙ ПРОЦЕСС → **ПОТЕРИ**

РЕШЕНИЕ

Система сама предсказывает спрос и вызывает транспорт — до того как склад переполнится

1

Прогноз спроса

На 7 дней и 5 часов вперёд с точностью, достаточной для автоматизации решений

2

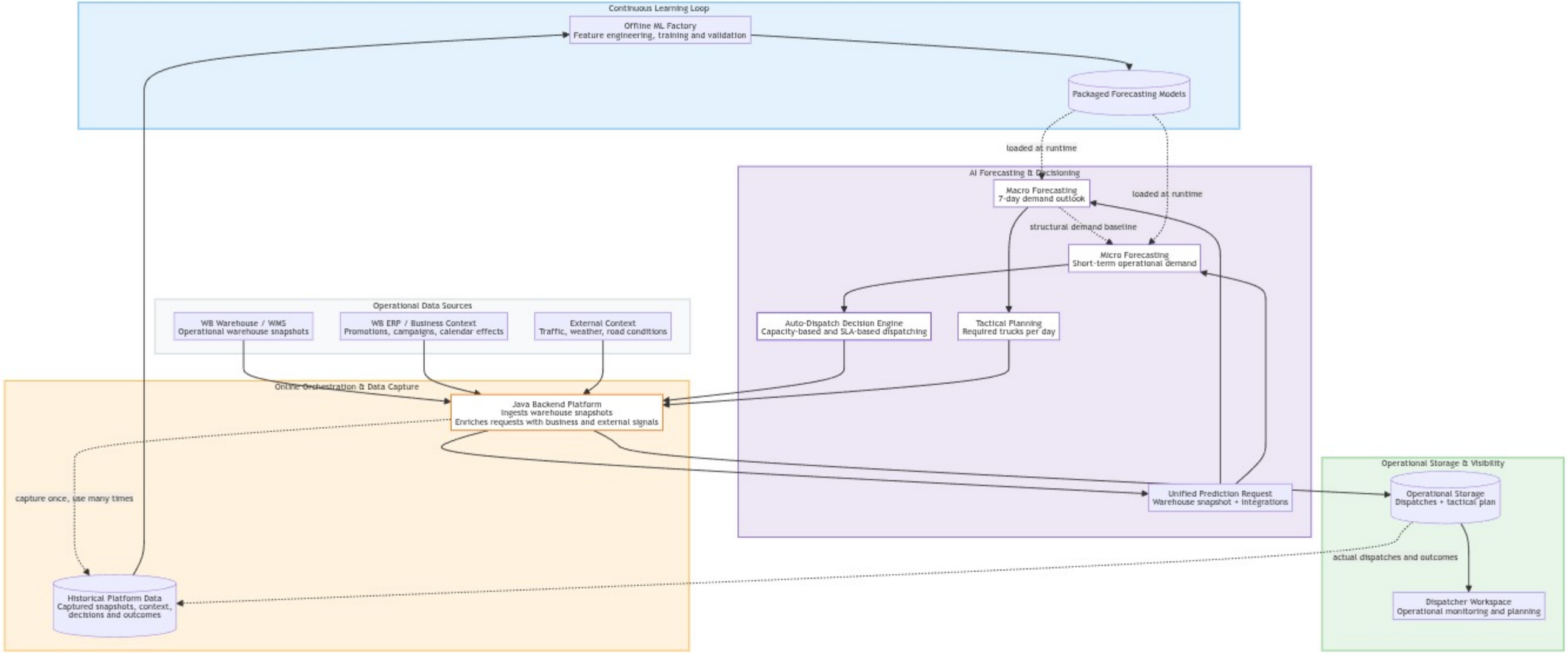
Dispatch-заявки

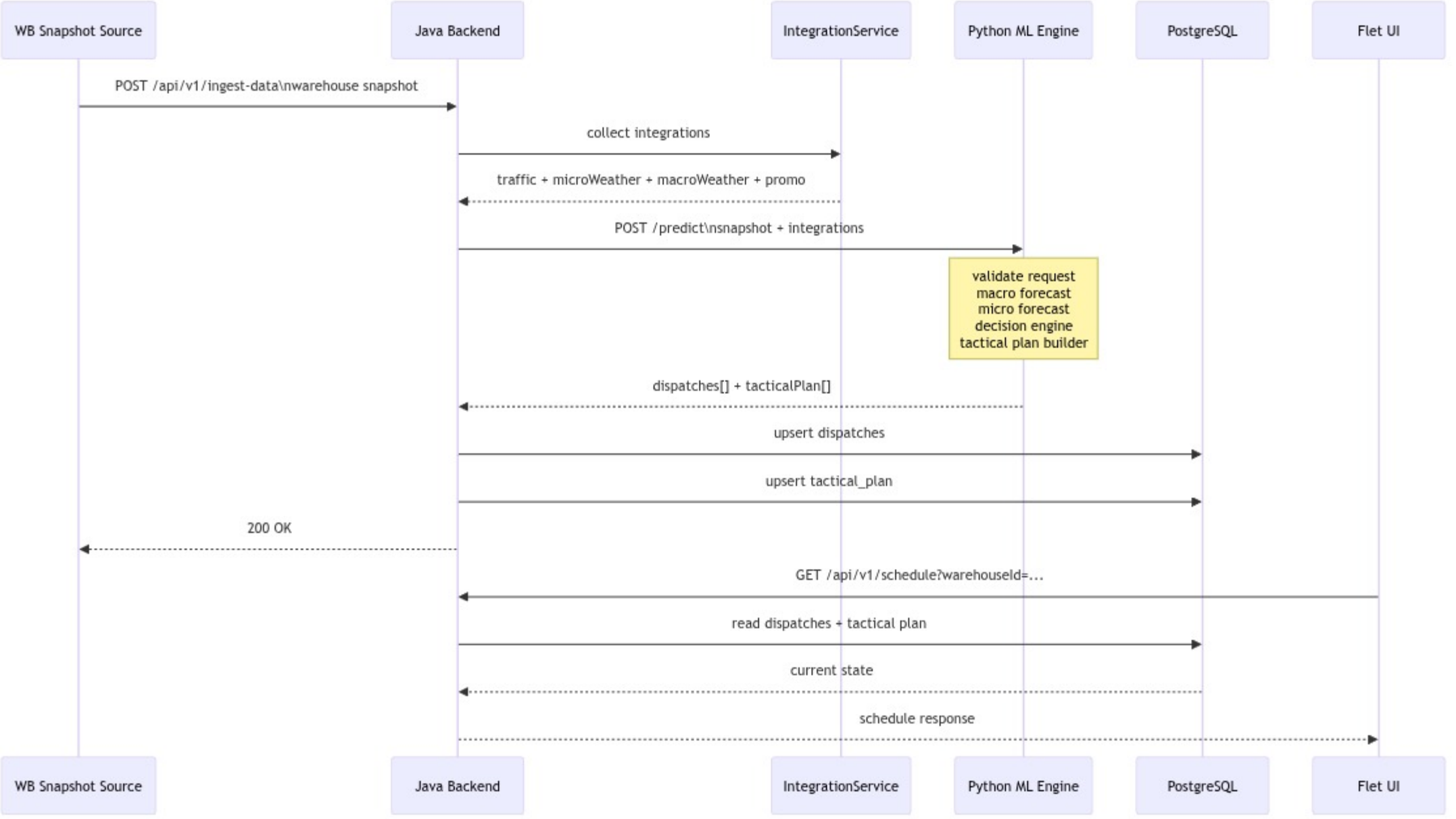
Автоматическое формирование по правилам SLA — без участия диспетчера

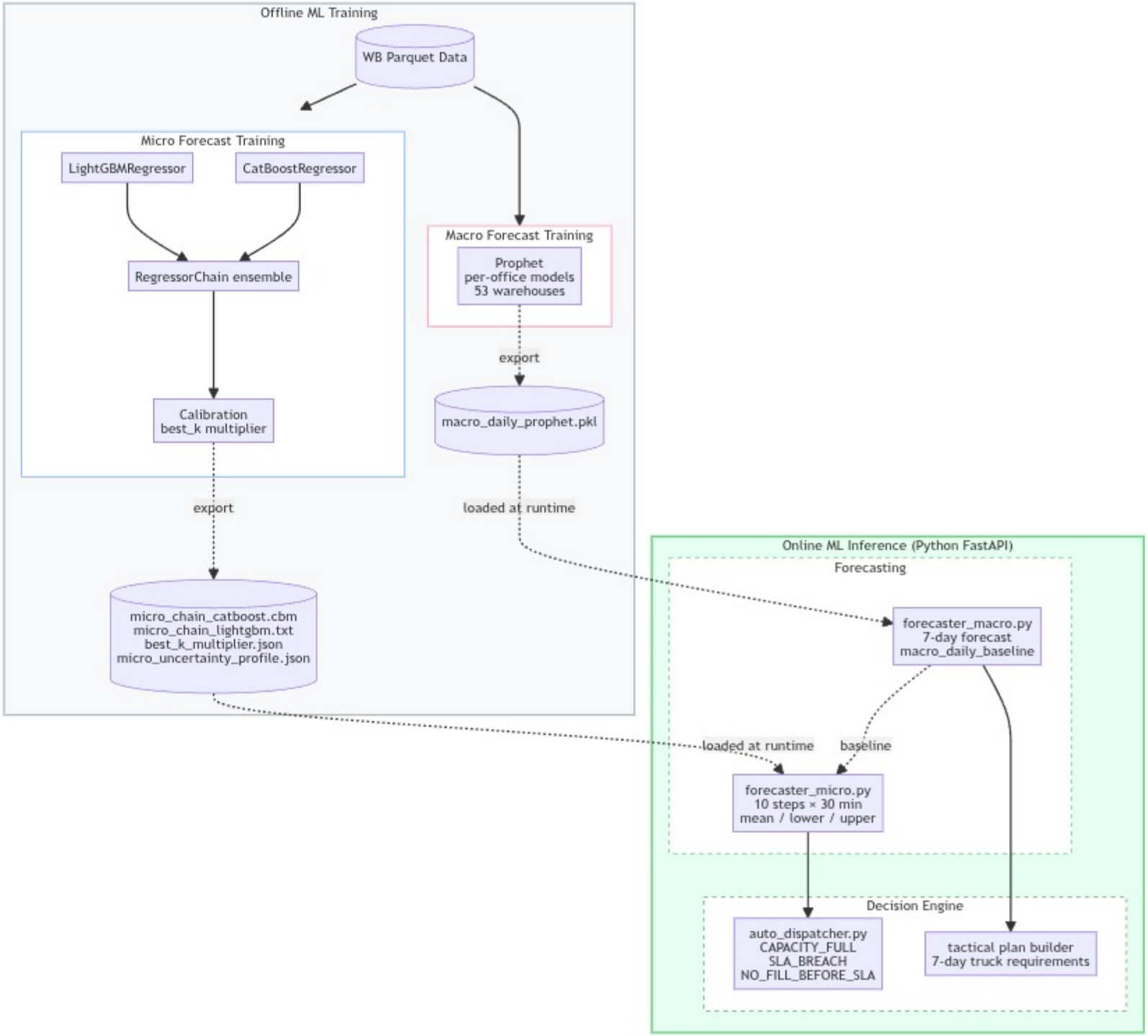
3

Production-ready

Полный стек в Docker — разворачивается за минуты, готов к продакшену


Continuous Learning Loop
Offline ML Factory
Feature engineering, training and validation
Packaged Forecasting Models
loaded at runtime
AI Forecasting & Decisioning
Macro Forecasting
7-day demand outlook
loaded at runtime
structural demand baseline
Micro Forecasting
Short-term operational demand
Auto-Dispatch Decision Engine
Capacity-based and SLA-based dispatching
Tactical Planning
Required trucks per day
Operational Data Sources
WB Warehouse / WMS
Operational warehouse snapshots
WB ERP / Business Context
Promotions, campaigns, calendar effects
External Context
Traffic, weather, road conditions
Online Orchestration & Data Capture
Java Backend Platform
Ingests warehouse snapshots
Enriches requests with business and external signals
capture once, use many times
Historical Platform Data
Captured snapshots, context, decisions and outcomes
Unified Prediction Request
Warehouse snapshot + integrations
Operational Storage & Visibility
Operational Storage
Dispatches + tactical plan
actual dispatches and outcomes
Dispatcher Workspace
Operational monitoring and planning


WB Snapshot Source
Java Backend
IntegrationService
Python ML Engine
PostgreSQL
Flet UI
POST /api/v1/ingest-data\nwarehouse snapshot
collect integrations
traffic + microWeather + macroWeather + promo
POST /predict\nsnapshot + integrations
validate request
macro forecast
micro forecast
decision engine
tactical plan builder
dispatches[] + tacticalPlan[]
upsert dispatches
upsert tactical_plan
200 OK
GET /api/v1/schedule?warehouseId=...
read dispatches + tactical plan
current state
schedule response
WB Snapshot Source
Java Backend
IntegrationService
Python ML Engine
PostgreSQL
Flet UI


Offline ML Training
WB Parquet Data
Micro Forecast Training
LightGBMRegressor
CatBoostRegressor
RegressorChain ensemble
Calibration
best_k multiplier
export
micro_chain_catboost.cbm
micro_chain_lightgbm.txt
best_k_multiplier.json
micro_uncertainty_profile.json
Macro Forecast Training
Prophet
per-office models
53 warehouses
export
macro_daily_prophet.pkl
loaded at runtime
Online ML Inference (Python FastAPI)
Forecasting
forecaster_macro.py
7-day forecast
macro_daily_baseline
loaded at runtime
baseline
forecaster_micro.py
10 steps × 30 min
mean / lower / upper
Decision Engine
auto_dispatcher.py
CAPACITY_FULL
SLA_BREACH
NO_FILL_BEFORE_SLA
tactical plan builder
7-day truck requirements

Двухуровневое прогнозирование

MACRO

Prophet

Модель временных рядов Facebook

Горизонт 7 дней

Недельный прогноз спроса

53 модели

Индивидуально по каждому складу

MICRO

CatBoost + LightGBM

Ансамбль градиентного бустинга

10 × 30 мин

Гранулярность 5 часов вперёд

Калибровка

Isotonic regression на предсказаниях

DECISION

- **CAPACITY_FULL**
  Склад достигнет предела
- **SLA_BREACH**
  Риск нарушения SLA
- **NO_FILL_BEFORE_SLA**
  Невозможно пополнить вовремя

Стратегический уровень

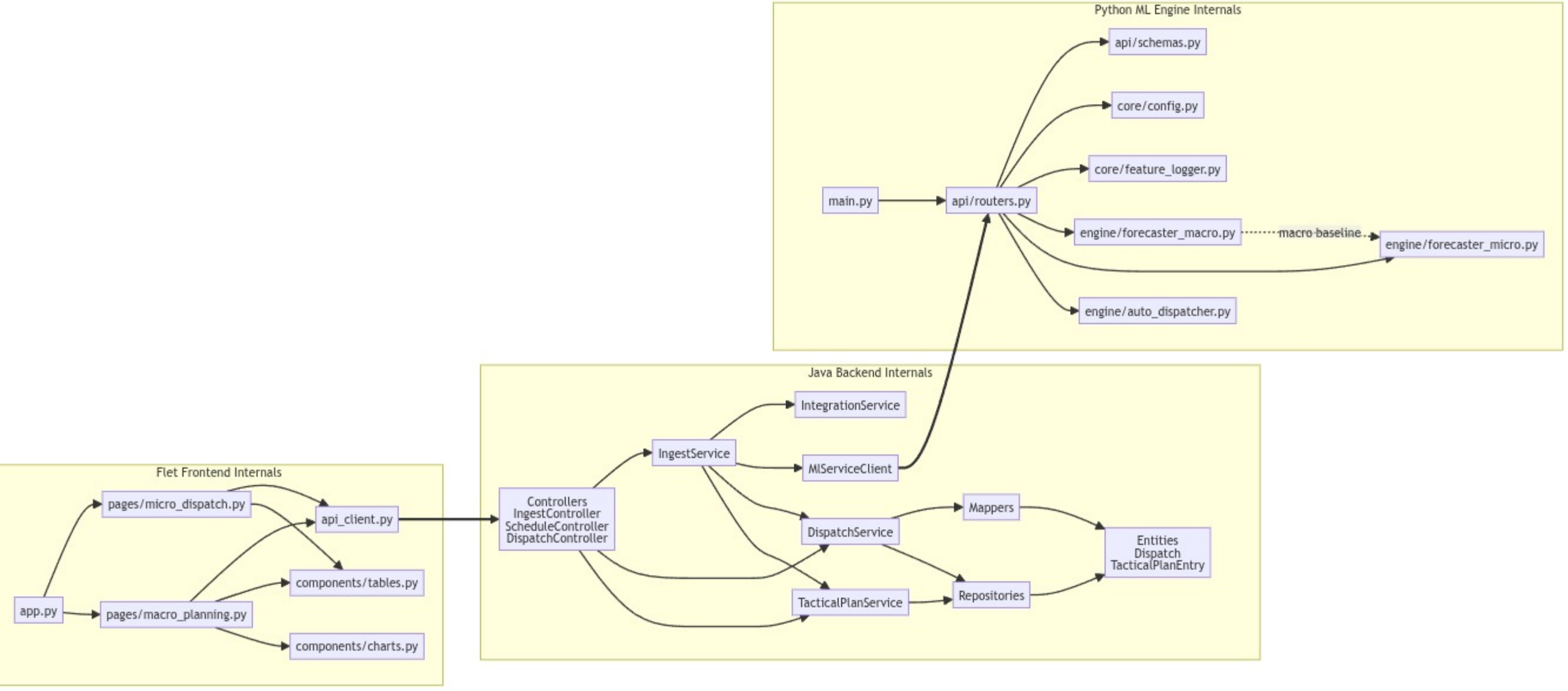
Планирование ресурсов и мощностей на неделю вперёд. Сезонность, тренды, праздники.

Операционный уровень

Реактивная подстройка каждые 30 минут. 200+ фичей, включая макро-прогноз.

Бизнес-сигналы

Три типа алертов запускают автоматические действия в системе управления.


Python ML Engine Internals
api/schemas.py
core/config.py
core/feature_logger.py
main.py
api/routers.py
engine/forecaster_macro.py
macro baseline
engine/forecaster_micro.py
engine/auto_dispatcher.py
Java Backend Internals
IntegrationService
IngestService
MlServiceClient
Controllers
IngestController
ScheduleController
DispatchController
Mappers
DispatchService
Entities
Dispatch
TacticalPlanEntry
TacticalPlanService
Repositories
Flet Frontend Internals
pages/micro_dispatch.py
api_client.py
components/tables.py
app.py
pages/macro_planning.py
components/charts.py

Production-ready стек

Backend

- Java 21
- Spring Boot 3
- PostgreSQL
- Flyway

ML Engine

- Python 3.11
- FastAPI
- CatBoost
- LightGBM
- Prophet
- Polars

Frontend

- React
- Vite
- nginx

Инфраструктура

- Docker Compose
- GitHub Actions CI/CD

Live демонстрация

Смотрим вживую

1. `make up`
   весь стек за одну команду
2. `POST /api/v1/ingest-data`
   ML создаёт заявки автоматически
3. `GET /api/v1/schedule`
   расписание рейсов с объёмами и типами ТС
4. `localhost:3000`
   дашборд диспетчера

Сделано за хакатон

- √ Java API
- √ Python ML Engine
- √ Prophet macro
- √ CatBoost+LightGBM micro
- √ Decision Engine
- √ PostgreSQL
- √ React фронтенд
- √ Docker Compose
- √ CI/CD
- √ **94 теста зелёных**

Что дальше

1. Реальные API погоды и пробок
   Замена моков на живые данные
2. Переобучение на полном датасете
   Полный исторический массив
3. Метрики качества в реальном времени
   Grafana + Prometheus мониторинг

10	94	3
компонентов	теста пройдено	языка

SealsTeam

https://github.com/Ronshin-Vsevolod/WBLogistic_SealsTeam.git